Harness Resilience: From LLM Availability to Toolchain Continuity in Agentic AI Engineering

John Cheung

College of Professional and Continuing Education

Abstract

By early 2026, the practical value of artificial intelligence in software engineering is no longer determined solely by the reliability of the underlying large language model (LLM). Increasingly, productivity depends on an execution harness: the surrounding runtime of prompts, tools, sandboxes, permissions, memory, repositories, connectors, model-routing policies, observability, and user-interface affordances that turn a model into an operational coding or general-purpose agent. Popular systems such as Codex, Claude Code, Gemini CLI, Cursor-like development environments, and general agents such as Manus demonstrate that the "AI product" is not only a model but a socio-technical toolchain. This paper argues that AI resilience must therefore be extended into *harness resilience*: the ability of an engineering team to continue safe and effective work when a preferred AI agent harness changes behavior, loses access, degrades, is rate-limited, is withdrawn from a plan, suffers a security incident, or becomes unavailable. Using an illustrative X.com post by Peter Gostev about a Claude Code user being humorously shown as opted out after a code-quality review, this paper motivates a broader resilience agenda: portability of context, tool abstraction, reproducible agent workflows, fallback harnesses, audit trails, and human recovery procedures. The paper contributes a reference architecture, a failure-mode taxonomy, measurable resilience objectives, a connector threat model, and an evaluation plan that turns the concept from a practice guide into a testable engineering framework.

Keywords: AI resilience, agentic AI, harness resilience, coding agents, Codex, Claude Code, Manus, MCP, AI engineering, software development lifecycle, provenance, agent testing

1 Introduction

The first wave of enterprise AI resilience focused on model-level questions: whether an LLM was accurate, safe, available, cost-effective, and compliant. This was natural when most use cases were chat-based or retrieval-augmented generation systems. However, by early 2026, the center of gravity has shifted. AI systems used by engineers increasingly act through rich harnesses that read codebases, edit files, run commands, call tools, coordinate multi-step tasks, and integrate with local or remote development environments.

In this setting, resilience cannot be limited to the LLM. A powerful model accessed through a fragile harness may be less useful than a slightly weaker model embedded in a reliable, auditable, portable, and well-governed workflow. The practical unit of dependency is no longer "the model" but the *model-harness combination*.

This paper defines a harness as the operational layer that surrounds a foundation model and enables it to act. In coding and agentic tools, this layer commonly includes system prompts, repository context,

https://doi.org/10.33774/coe-2026-4f53g **ORCID:** https://orcid.org/0009-0007-9777-4393 Content not peer-reviewed by Cambridge University Press. **License:** CC0 1.0

command execution, tool definitions, permissions, connector protocols such as MCP, task orchestration, rate limits, feature flags, logging, review, rollback, and user-interface affordances.

The key claim is simple: *AI engineering teams need business-continuity planning for their AI harnesses*. If a harness disappears tomorrow, changes its behavior, loses a connector, changes its policy, or becomes untrusted, the team should still be able to build, debug, test, and ship software.

1.1 Contributions

This position paper makes five contributions:

1. defines harness resilience as a distinct layer between model reliability and application reliability;
2. proposes a reference architecture and boundary criteria for the model, harness, application, and governance layers;
3. introduces measurable objectives for functional, context, tool, governance, and human resilience;
4. provides a connector-oriented threat model and failure-mode taxonomy;
5. outlines an evaluation plan using golden tasks, failure injection, provenance metrics, and cost-latency analysis.

2 Background: From LLMs to Harnesses

Agentic coding tools in 2025–2026 demonstrate a major design transition. Claude Code is commonly used as an agentic coding tool that can read a codebase, edit files, run commands, and integrate with development tools [1]. Gemini CLI similarly popularizes an AI agent in the terminal and documents a reason-and-act style loop using built-in tools and local or remote MCP servers [2]. Codex has expanded from a coding model brand into a broader software engineering assistant and enterprise workflow [3]. General agents such as Manus further popularized the idea that an AI system should bridge ideation and execution rather than merely answer questions.

These systems are not interchangeable chat windows. They embody distinct assumptions about how context is collected and compacted, how files are edited, how shell commands are approved, how secrets are protected, how plans are revised, how sub-agents are delegated, how tool failures are surfaced, and how much control the user retains. Thus, the harness is a source of productivity, but also a source of lock-in and fragility.

3 Boundary and Reference Architecture

A precise boundary is needed because the terms “harness”, “agent”, and “application” are often used interchangeably. In this paper, the **model** is the statistical inference service or local model. The **harness** is the orchestration and execution substrate that packages context, invokes the model, exposes tools, manages permissions, records traces, and applies policy. The **application** is the user-facing product or business workflow that embeds the harness. The **governance layer** is the organizational control plane that sets data, access, procurement, incident, and audit requirements.

https://doi.org/10.33774/coe-2026-4f53g **ORCID:** https://orcid.org/0009-0007-9777-4393 Content not peer-reviewed by Cambridge University Press. **License:** CC0 1.0

Layer	Boundary and responsibility
Application / IDE / CLI	User-facing workflow, user interaction, product-specific screens, and business task framing.
Agent harness	Planning loop, context assembly, tool exposure, policy enforcement, approvals, execution control, traces, and fallback routing.
Model provider	LLM or SLM inference service, model weights, model endpoint, and model-level safety behavior.
Tools and connectors	MCP servers, shell, Git, CI, APIs, browsers, databases, and external systems invoked by the harness.
Project memory and provenance	AGENTS.md, runbooks, context manifests, tool schemas, logs, commits, and trace stores.
Governance layer	Security, privacy, procurement, audit, data classification, change notice, and incident response.

Table 1: Reference architecture for model-harness-application resilience.

The boundary criterion is operational: if replacing a component requires changing context packaging, tool permissions, workflow traces, approval gates, or user interaction patterns, that component is part of the harness or tightly coupled to it. This distinction matters because a model can be healthy while the harness is degraded, or a harness can be portable while the application remains vendor-specific.

3.1 Boundary Contract Checklist

For automated governance, each component can publish a small boundary contract. The contract declares: (i) inputs and outputs; (ii) state it can read or mutate; (iii) external tools or connectors it can call; (iv) credentials and privileges it can access; (v) trace fields it emits; (vi) failure modes and fallback path; and (vii) replacement interface. A component is classified as harness-critical when it satisfies any of four tests: replacing it changes tool authorization, changes context assembly, changes state mutation, or changes audit evidence. This contract converts the intuitive harness/application distinction into a checklist that can be reviewed in architecture governance and CI.

4 The Early 2026 Status of Harness Dependence

By early 2026, harness dependence has become visible in three ways.

4.1 Adoption at Scale

Coding agents have moved from experimental tools to mainstream engineering infrastructure. Public discussion around Codex, Claude Code, Gemini CLI, Manus, and related products shows that agentic coding tools are no longer side utilities; they are becoming part of organizational software delivery systems.

4.2 Product-Level Degradation

The Claude Code performance discussions in 2026 illustrate why harness resilience matters [4]. In an agentic product, user experience can degrade even when the base model remains strong. A change in prompt policy, cache behavior, context packaging, tool execution, default reasoning level, or user-interface workflow may reduce practical reliability.

For engineering teams, this means that monitoring only model benchmarks is insufficient. Teams should monitor the end-to-end harnessed workflow: instruction loading, repository reading, tool invocation, shell execution, patch quality, test execution, reviewability, and rollback.

https://doi.org/10.33774/coe-2026-4f53g **ORCID:** https://orcid.org/0009-0007-9777-4393 Content not peer-reviewed by Cambridge University Press. **License:** CC0 1.0

4.3 Protocol and Connector Risk

The rise of MCP-style tool integration makes harnesses more powerful but also increases the attack surface. MCP standardizes how agents connect to external tools and data sources [5]. This improves interoperability, but it also creates dependency on connector security, tool schemas, command execution boundaries, and permission models. A harness that can call tools can also misuse tools, over-trust tool output, leak context, or execute unsafe operations if its governance layer is weak.

5 Illustrative Case: When a Harness Relationship Breaks

A post on X.com by Peter Gostev, AI Capability Lead of Arena.ai, dated 26 April, presents a humorous Claude Code scenario in which the user is shown as being opted out after a review of code quality [6]. This paper does not treat the post as authenticated evidence of an actual vendor policy. Instead, it is useful as a cultural signal: developers increasingly perceive AI coding tools as active participants in the development workflow, not passive utilities.

The joke works because it exposes a real anxiety. If a coding agent learns from, evaluates, stores, filters, or governs user work, then the user is dependent not only on model quality but also on the rules and policies of the surrounding harness. A vendor may change who can access a tool, which plan includes it, whether user data is retained, how code is routed or stored, which tools are permitted, or whether a workflow is considered abusive, risky, or unsupported.

The resilience question is therefore: *what can an AI engineer do when the harness is no longer available?*

6 Related Work

Harness resilience sits at the intersection of software reliability engineering, agent testing, provenance, security governance, and context management. AgentAssay gives a statistical foundation to the "golden task" idea by treating agent regression testing as a stochastic process with pass, fail, and inconclusive outcomes, behavioral fingerprints, and sequential probability ratio testing (SPRT) to control error and trial cost [7]. More general work on AI-agent reliability argues that a single success rate hides important operational failures and decomposes reliability into consistency, robustness, predictability, and safety [8]. Harness resilience complements this line of work by focusing not only on whether an agent succeeds, but whether the organization can recover when the harness, connector, memory, or policy layer changes.

Trust, Risk, and Security Management (TRiSM) for agentic multi-agent systems emphasizes governance, explainability, ModelOps, and privacy/security [9]. Harness resilience can be read as the business-continuity counterpart to TRiSM: it converts trust and risk concerns into operational targets such as time-to-fallback, provenance coverage, connector replacement ratio, and recovery time.

Auditability and provenance work provides a second foundation. Claim-level auditability work introduces provenance metrics such as provenance coverage, soundness, contradiction transparency, and audit effort for deep research agents [10]. R-LAM proposes structured action schemas, deterministic execution policies, and explicit provenance tracking for replayable action workflows [11]. For engineering harnesses, these ideas can be adapted from research claims to software actions: every file edit, tool call, command, approval, and generated explanation should be traceable to a context state, model invocation, and human decision. W3C PROV defines a domain-agnostic model for entities, activities, and agents [12]; OpenTelemetry traces represent operations through spans that can be exported to observability backends [13]. These standards offer practical building blocks for harness trace schemas.

https://doi.org/10.33774/coe-2026-4f53g **ORCID:** https://orcid.org/0009-0007-9777-4393 Content not peer-reviewed by Cambridge University Press. **License:** CC0 1.0

Context portability is addressed by Git-Context-Controller, which treats agent memory as a persistent, versioned hierarchy inspired by Git operations such as commit, branch, merge, and context [14]. The Agentic File System abstraction similarly treats context artifacts as governed, mounted, traceable resources [15]. Lore proposes structured knowledge atoms embedded in Git commit metadata, offering a lightweight mechanism for preserving implementation rationale with code changes [16]. Context-compression and memory-action systems, including empirical work on repository-level context compression and autonomous memory operations, are also relevant because they move context selection and forgetting into the harness itself [17, 18]. These modules can improve latency and token cost, but they should be governed as resilience-sensitive components because they affect what the agent remembers, omits, and can export.

Systems work on persistent multi-agent orchestration, such as TheBotCompany, demonstrates milestone-driven strategy, execution, and verification loops for longer software-development tasks [19]. SWE-Skills-Bench shows that injected software-engineering skills may have limited or even negative marginal value when mismatched with project context, which supports the need for harness-level compatibility checks, versioned skill registries, and regression gates [20]. Automated Self-Testing further suggests evidence-driven release gates with promote, hold, and rollback decisions for LLM applications [21]. Together, these works make harness resilience measurable rather than only aspirational.

7 Defining Harness Resilience

Harness resilience is the ability of a team to maintain acceptable engineering productivity, safety, and governance when an AI harness fails, changes, or disappears. It has five dimensions.

7.1 Functional Resilience

The team can still perform core tasks: code search, code generation, refactoring, test creation, debugging, documentation, migration, review, and release support.

7.2 Context Resilience

The team can preserve and reuse project context, including architecture notes, coding standards, repository maps, API contracts, testing instructions, deployment procedures, and known failure modes.

7.3 Tool Resilience

The team can switch between harnesses without rewriting every tool integration. This requires abstraction of tool schemas, stable command wrappers, documented permissions, and fallback execution paths.

7.4 Governance Resilience

The team can maintain auditability, security, privacy, and compliance even when moving between vendors or local alternatives.

7.5 Human Resilience

Engineers remain capable of understanding, reviewing, and completing work manually. The harness may accelerate engineering, but it must not become the only holder of operational knowledge.

https://doi.org/10.33774/coe-2026-4f53g **ORCID:** https://orcid.org/0009-0007-9777-4393 Content not peer-reviewed by Cambridge University Press. **License:** CC0 1.0

8 Measurable Objectives and Metric Protocols

Harness resilience should be stated as measurable objectives rather than general intent. Table 2 defines candidate KPIs with units, denominators, and recommended reporting practices. The aim is not to impose one universal score, but to make each team's continuity posture reproducible and comparable over time.

Dimension	KPI	Definition	Reporting Protocol
Functional	Time-to-fallback (TTF)	Minutes from primary-harness unavailability to productive continuation using a secondary harness or manual path.	Report median, p90, and maximum by task class; normalize by task complexity tier and team size.
Recovery	Mean time to harness recovery (MTTHR)	Time to restore the preferred harness or approve a substitute for production use.	Report per incident type with confidence intervals when repeated drills are available.
Context	Context exportability score (CES)	$CES = \frac{\sum w_i e_i}{\sum w_i}$ where $e_i \in \{0, 0.5, 1\}$ scores portability of each required context artifact.	Score against a manifest containing instructions, repo map, runbook, tool schemas, permissions, memory, examples, and validation commands; use two reviewers for human-judged items.
Tool	Connector replacement ratio (CRR)	Fraction of tool capabilities implemented through documented wrappers or standard APIs rather than vendor-internal calls.	Count by capability, not raw call volume; mark partial replacements as 0.5 and destructive operations separately.
Governance	Provenance coverage (PCov)	Fraction of agent actions with a complete trace record under the minimum schema in Table 3.	Report by action class: file edit, tool call, shell command, approval, retrieval, and memory update.
Security	Permission surface area (PSA)	Weighted exposure score $PSA = \sum s_j p_j$ where s_j is sensitivity and p_j is privilege breadth for each secret, API, or command.	Use a 1–5 sensitivity scale and record whether access is read-only, write, destructive, time-limited, and compartmentalized.
Human	Manual completion rate (MCR)	Fraction of golden tasks completed without the preferred harness within the agreed recovery window.	Report by task family and include intervention count, defect leakage, and developer confidence.
Cost	Resilience overhead ratio (ROR)	Additional cost or latency introduced by redundancy, sandboxing, logging, or repeated tests divided by baseline cost or latency.	Report separately for token cost, wall-clock latency, storage, and developer approval time.

Table 2: Formal KPI definitions for harness resilience.

A concrete CES example illustrates the intended scoring. Suppose a repository has eight required artifacts with equal weights: `AGENTS.md`, architecture map, test commands, deployment notes, tool schema manifest, permission manifest, memory export, and golden-task list. If five are directly portable, two require minor rewriting, and one is missing, then $CES = (5 + 2 \times 0.5 + 0)/8 = 0.75$. A regulated team may set a release threshold of $CES \geq 0.85$, $PCov \geq 0.95$ for production-impacting actions, and p90 TTF below one working day. A small research prototype may accept lower targets but should still record the denominator and scoring basis.

8.1 Minimum Provenance Trace Schema

Table 3 operationalizes provenance coverage. A trace is complete only when all mandatory fields for the action class are captured. The schema can be serialized as JSON, stored with CI logs, and mapped to W3C PROV and OpenTelemetry: the task, files, artifacts, and outputs are PROV entities; model calls, tool calls, and shell commands are PROV activities and OpenTelemetry spans; users, agents, services, and reviewers are PROV agents.

https://doi.org/10.33774/coe-2026-4f53g **ORCID:** https://orcid.org/0009-0007-9777-4393 Content not peer-reviewed by Cambridge University Press. **License:** CC0 1.0

Trace Field	Meaning	Completeness Rule
Trace and task ID	Stable identifiers linking the run, ticket, branch, and golden task.	Required for every action.
Context version	Hashes or versions of instructions, retrieved files, memory snapshot, and tool-schema manifest.	Required before every model call and tool call.
Model and harness version	Model name, provider, harness version, feature flags, prompt-policy version, and context-compression module.	Required for reproducibility and regression analysis.
Action record	Tool name, shell command, file edit, retrieval, memory update, or approval request.	Required for all state-changing actions; read-only retrievals may be sampled in low-risk workflows.
Permission record	Credential scope, sandbox identity, approval status, and policy decision.	Required for write, destructive, external, or secret-bearing actions.
Inputs and outputs	Redacted prompt digest, tool arguments, output hash, patch diff, test result, and error state.	Required; sensitive values should be hashed or redacted, not omitted.
Reviewer and disposition	Human reviewer, approval/rollback decision, and linked commit or pull request.	Required for production-impacting changes.

Table 3: Minimum trace schema for measuring provenance coverage.

8.2 Mapping to Reliability and TRiSM Dimensions

Table 4 relates harness KPIs to agent-reliability and TRiSM-style governance dimensions. This mapping helps avoid reinventing reliability terminology and enables joint reporting with model-evaluation dashboards.

Harness KPI	Reliability Dimension	TRiSM / Governance Link	Interpretation
TTF and MTTHR	Robustness under disruption	Lifecycle governance and ModelOps	Measures continuity when the harness or connector is perturbed.
CES and CRR	Robustness and predictability	Governance and interoperability	Measures whether context and tools can be moved without hidden coupling.
PCov	Safety and predictability	Explainability, audit, and accountability	Measures whether actions can be reconstructed and reviewed.
PSA	Safety	Security and privacy	Measures the exposure created by tool access and credential scope.
MCR	Human-centered resilience	Lifecycle governance	Measures whether expertise remains outside the harness.
ROR	Operational predictability	ModelOps and procurement	Measures the cost of controls relative to baseline workflow.

Table 4: Mapping harness-resilience KPIs to broader reliability and governance dimensions.

9 Failure Modes of AI Harnesses

Harness failures are not limited to outages. They include degradations in quality, governance, economics, and recoverability. Table 5 summarizes common modes.

https://doi.org/10.33774/coe-2026-4f53g **ORCID:** https://orcid.org/0009-0007-9777-4393 Content not peer-reviewed by Cambridge University Press. **License:** CC0 1.0

Failure Mode	Example	Resilience Response
Access loss	Tool removed from plan, account suspended, region unsupported	Maintain secondary harness and local workflow.
Performance regression	Prompt, cache, model-routing, or context change reduces quality	Run golden-task regression tests and compare behavioral fingerprints.
Connector failure	MCP server, GitHub, Jira, database, or CI connector unavailable	Use manual and API fallback paths; maintain connector health checks.
Security incident	Prompt injection, unsafe tool execution, credential exposure	Apply least privilege, sandboxing, dry-run verification, and approval gates.
Policy change	Data retention, training, privacy, or acceptable-use policy changes	Maintain data classification rules and a vendor-switch plan.
Cost shock	Rate limits, token pricing, or enterprise terms change	Use budget caps, routing alternatives, and local fallbacks for low-risk tasks.
Context lock-in	Knowledge trapped in proprietary memory or hidden project state	Externalize context into repository files, runbooks, commit metadata, and exportable traces.
Workflow opacity	Agent actions are hard to reproduce or audit	Use structured logs, span traces, provenance records, and commit-level traceability.

Table 5: Common harness failure modes and resilience responses.

10 Threat Model for Connectorized Harnesses

A connectorized harness expands the boundary of trust. The threat model should identify assets, adversaries, attack paths, controls, and measurable detection or containment outcomes. Table 6 gives a compact attack-tree view suitable for risk registers and failure-injection exercises.

Root Risk	Attack Path	Minimum Control	Test Metric
Unsafe tool use	Malicious tool schema or poisoned README induces a destructive command.	Tool allow-list, typed schemas, sandbox, dry-run mode, approval for destructive calls.	True positive rate for seeded unsafe-call attempts; false approval rate must be near zero for critical actions.
Secret leakage	Credentials appear in prompts, logs, tool outputs, generated code, or traces.	Scoped ephemeral credentials, redaction, secret scanners, compartmentalized service accounts.	Leakage detection rate, false negatives in seeded canary secrets, and maximum credential lifetime.
Connector compromise	MCP or API connector returns adversarial output or performs unexpected side effects.	Connector signing, capability scoping, outbound egress control, health checks, replayable logs.	Containment rate under mocked malicious connector; mean time to revoke or replace connector.
Retrieval contamination	Tickets, documents, or web pages contain prompt injection that overrides policy.	Instruction hierarchy, retrieval labeling, untrusted-content delimiters, policy-as-code checks.	Refusal/approval accuracy for injected retrieval corpus; false positive rate on benign content.
Approval bypass	UI or automation path misclassifies high-risk operation as routine.	Risk classifier, mandatory human approval, two-person rule for high-impact production actions.	Missed escalation rate and approval latency overhead.
Memory corruption	Learned memory or compression module deletes, distorts, or locks in critical context.	Versioned context manifest, reversible compression, memory diff review, retention policy.	Context reconstruction rate and CES delta after compression or memory update.

Table 6: Connector-oriented attack paths, controls, and measurable tests.

Likelihood and severity should be scored separately. A lean team can use a 1–5 ordinal scale: likelihood reflects observed frequency or ease of exploitation, while severity reflects confidentiality, integrity, availability, safety, and compliance impact. The residual-risk score after controls should be tracked across

https://doi.org/10.33774/coe-2026-4f53g **ORCID:** https://orcid.org/0009-0007-9777-4393 Content not peer-reviewed by Cambridge University Press. **License:** CC0 1.0

releases. This does not replace a full STRIDE or PASTA analysis, but it gives AI engineering teams a practical connector-specific starting point.

The testable question is not whether a harness is "secure" in general; it is whether each high-impact action has a defined boundary, observable trace, and recoverable failure mode. A minimal security exercise should include seeded prompt injections in repository files, tool-output manipulation, canary secrets, a mocked connector outage, and at least one destructive command that must be blocked or escalated.

11 Practical Actions for AI Engineers

11.1 Maintain a Harness-Agnostic Project Context

Every repository using AI coding agents should contain a vendor-neutral context package. This may include:

- `AGENTS.md` or equivalent project instructions;
- architecture overview and repository map;
- development setup and test commands;
- coding conventions and security constraints;
- deployment notes and rollback procedures;
- examples of good changes and known anti-patterns;
- exported tool schemas and connector permissions.

When a harness is unavailable, the same context can be loaded into another agent or used by humans. Appendix-style artifacts that should be released with a practical implementation include an extttAGENTS.md template, a context-manifest schema, a fallback checklist, and a KPI calculator.

11.2 Separate Model, Harness, and Tool Layers

Teams should avoid designing workflows in which the model, harness, and tools are inseparable. A resilient architecture separates the model provider, agent runtime, tool protocol, execution environment, project memory, and audit/governance layer. This makes replacement possible. For example, a team may use Claude Code for one class of tasks, Codex for another, Gemini CLI for open-source terminal workflows, and a local model for restricted code review, provided that common instructions and tools are portable.

11.3 Build an Agent Runbook

A harness resilience runbook should answer: which tasks depend on which AI tools; what the manual and secondary AI fallbacks are; what data is allowed in each tool; how secrets are protected; how agent-generated changes are reviewed; how an agent action can be reproduced; and who decides when to disable a harness.

https://doi.org/10.33774/coe-2026-4f53g **ORCID:** https://orcid.org/0009-0007-9777-4393 Content not peer-reviewed by Cambridge University Press. **License:** CC0 1.0

11.4 Use Golden Tasks for Regression Testing

Teams should maintain a small benchmark of internal "golden tasks" that represent real engineering work, such as fixing a known bug, adding a small endpoint, refactoring without behavior change, writing unit tests, explaining a legacy subsystem, or detecting a security issue in a pull request. To avoid anecdotal testing, each task should define expected artifacts, pass/fail/inconclusive criteria, trace fields, and a token budget. Sequential testing methods such as AgentAssay can be used where repeated stochastic runs are necessary [7].

11.5 Capture Provenance and Traces

Harness actions should produce trace records that can be stored alongside CI logs and commits. A minimal record contains the task ID, context version, model/harness version, tool calls, files changed, commands executed, approvals, test results, and final human reviewer. OpenTelemetry spans can represent the temporal execution trace, while W3C PROV-style entities, activities, and agents can represent causality and responsibility [12, 13].

11.6 Design for Human Review and Rollback

Harness resilience depends on ordinary software engineering discipline: small commits, branch isolation, code review, test coverage, CI gates, reproducible environments, and rollback procedures. The more autonomous the harness, the stronger these controls must be.

11.7 Control Tool Permissions

Agent tools should follow least privilege. A coding agent may need to read a repository and run tests, but it rarely needs unrestricted production credentials. Sensitive operations should require explicit approval, separate identities, sandboxed environments, dry-run modes, or policy-as-code checks.

11.8 Keep a Local or Open Alternative

A resilient AI engineering team should maintain at least one fallback path that does not depend on a single vendor account. This may be an open-source CLI harness, a locally hosted model for basic coding support, or a minimal scriptable workflow that provides repository search, context packing, and patch generation. The fallback does not need to match the primary harness in quality; it only needs to preserve continuity during incidents.

12 Evaluation Plan and Initial Study Design

A stronger validation program should combine controlled experiments with field case studies. The evaluation should instantiate the KPIs rather than merely list them.

12.1 Cross-Harness Golden-Task Protocol

A minimal study can use 20–40 tasks sampled from three classes: code modification, debugging/test repair, and repository understanding. Each task should be pinned to a repository commit, include acceptance criteria, and define deterministic verification where possible. The study should run each task under: primary harness, secondary harness, and manual fallback. For stochastic harnesses, repeated runs can be stopped

https://doi.org/10.33774/coe-2026-4f53g **ORCID:** https://orcid.org/0009-0007-9777-4393 Content not peer-reviewed by Cambridge University Press. **License:** CC0 1.0

early using SPRT-style sequential testing when the difference is clear, while inconclusive outcomes are reported rather than forced into pass/fail categories.

For each task, report: success rate, TTF, number of developer interventions, test pass rate, defect leakage after review, PCov, token cost, wall-clock latency, and ROR. Report medians and p90 values rather than only means, because recovery workflows often have heavy-tailed delays.

12.2 Failure Injection

Failure injection should simulate at least five events: access loss, connector outage, rate limit, prompt-policy change, and tool-permission denial. For each event, the experiment records baseline impact, fallback path used, TTF, MTTHR, error rate, provenance completeness, and whether the runbook was sufficient. A successful drill is not one in which nothing fails; it is one in which the team detects the failure, moves to an approved fallback, preserves auditability, and completes or safely stops the task.

12.3 Security and Provenance Exercises

Security exercises should seed prompt-injection payloads in issues, README files, dependency documentation, and tool outputs; introduce canary secrets; and mock a malicious connector that attempts unexpected side effects. Measurements include true positives, false positives, false negatives, containment rate, approval latency, and rollback success. Provenance audits should sample completed agent actions and compute PCov using Table 3. A trace should be considered incomplete if it lacks context version, tool arguments, permission record, output hash, or reviewer disposition for a state-changing action.

12.4 Synthetic Example of Metric Reporting

A small pilot could report the following synthetic format: 30 golden tasks, two harnesses, one manual fallback, three injected incidents. Results might be presented as "primary harness p90 TTF under connector outage = 3.8 hours; secondary harness p90 TTF = 1.1 hours after runbook update; PCov improved from 0.62 to 0.94 after trace instrumentation; ROR = 1.18x token cost and 1.07x wall-clock latency." Such a table would not prove universal validity, but it would ground the framework and make future replication possible.

12.5 Cost, Latency, and Developer Experience

Resilience controls should be right-sized. High-criticality workflows should record complete traces, enforce approvals, and test redundant harnesses. Low-risk exploratory coding can use sampled tracing and lightweight fallbacks. Teams should track developer approval time, false-positive security blocks, and survey-based friction scores. A control that improves PCov but doubles cycle time for low-risk changes may be inappropriate; the same control may be mandatory for production-impacting automation.

This evaluation plan does not require every team to publish academic benchmarks. It requires operational evidence that the harness can fail without stopping the engineering organization.

https://doi.org/10.33774/coe-2026-4f53g **ORCID:** https://orcid.org/0009-0007-9777-4393 Content not peer-reviewed by Cambridge University Press. **License:** CC0 1.0

13 A Harness Resilience Maturity Model

Level	Description
0: Ad hoc	Engineers use AI tools individually with no shared standards.
1: Documented	Basic project instructions and allowed-use policies exist.
2: Portable	Context files, prompts, development commands, and tool schemas are reusable across tools.
3: Governed	Permissions, logs, review gates, data controls, and provenance records are enforced.
4: Redundant	Multiple harnesses and manual fallbacks are tested for critical workflows.
5: Antifragile	Harness incidents improve the workflow through regression tests, failure injection, runbook updates, and architecture hardening.

Table 7: A maturity model for harness resilience.

14 Implications for Enterprises

For enterprises, harness resilience should become part of AI governance and business continuity planning. Procurement should evaluate not only model accuracy and price, but also exportability of project memory, support for open standards, audit logging, administrative controls, sandboxing options, data residency, incident communication, contractual change notice, and the ability to run controlled regression tests.

Controls should be right-sized. Redundancy and provenance carry cost, latency, and developer-experience overhead. High-criticality workflows, regulated data, and production-impacting automation justify stronger gates. Low-risk exploratory coding may use lighter logging and fewer repeated tests. The goal is not to slow every developer; it is to prevent a fragile harness dependency from becoming a single point of organizational failure.

15 Discussion

The phrase "AI resilience" should now include three layers:

1. **Model resilience**: quality, safety, latency, cost, and availability of the LLM.
2. **Application resilience**: reliability of the AI-enabled product or workflow.
3. **Harness resilience**: continuity of the execution layer that turns model output into action.

The third layer is becoming increasingly important. Coding agents and general agents are effective precisely because they are not just chatbots. They hold context, execute tools, modify state, and shape engineering practice. Therefore, their failure is not merely an inconvenience; it can interrupt the software delivery lifecycle.

The Gostev X.com post is humorous, but the underlying lesson is serious. AI engineers should not ask only, "Which model is best?" They should ask, "Can our team still work if this harness changes tomorrow?"

16 Limitations

This paper is primarily a position paper and practice framework. It does not yet report a controlled empirical study, multi-organization case evidence, or statistically powered comparison of harnesses. Its proposed KPIs and evaluation plan require calibration in real engineering environments. Future work should publish open templates, reference harnesses, reproducible golden-task suites, and anonymized incident studies that measure effect sizes, costs, and failure recovery outcomes.

https://doi.org/10.33774/coe-2026-4f53g **ORCID:** https://orcid.org/0009-0007-9777-4393 Content not peer-reviewed by Cambridge University Press. **License:** CC0 1.0

17 Recommendations

We recommend that AI engineering teams adopt the following minimum practices:

1. Maintain a vendor-neutral project instruction file and context package.
2. Keep architecture and operational knowledge outside proprietary agent memory.
3. Define measurable KPIs such as TTF, MTTHR, context exportability, and provenance coverage.
4. Test at least two AI harnesses on critical development tasks.
5. Record agent-generated changes through normal version control and trace logs.
6. Require human review for production-impacting modifications.
7. Sandbox command execution and restrict secrets.
8. Maintain a manual fallback for release-critical workflows.
9. Track harness incidents separately from model incidents.
10. Rehearse a "no preferred AI tool" day once per quarter for high-dependency teams.

18 Conclusion

The current agentic AI landscape marks a transition from model-centric AI adoption to harness-centric AI operations. Codex, Claude Code, Manus, Gemini CLI, and similar tools show that the productivity frontier lies in systems that can plan, act, call tools, and manage context. However, this also means that engineering teams inherit new dependencies.

Harness resilience is the discipline of surviving those dependencies. It asks teams to make context portable, tools replaceable, actions auditable, permissions constrained, and humans capable of recovery. The most resilient organizations will not be those that depend on no AI tools. They will be those that can use powerful AI harnesses without becoming trapped by them.

References

[1] Anthropic. *Claude Code Overview*. Claude Code documentation, accessed early 2026.

[2] Google Developers. *Gemini CLI Documentation*. Google Developers documentation, accessed early 2026.

[3] OpenAI and public reporting. *Codex and Agentic Coding Adoption*. 2026.

[4] Public reporting and developer discussion. *Claude Code Performance and Product-Level Reliability*. 2026.

[5] Model Context Protocol. *Specification*. MCP documentation, 2025.

[6] Peter Gostev. *X.com post on Claude Code and data sharing*. 26 April 2026. Available at: `https://x.com/petergostev/status/2048154801926447382?s=46`.

https://doi.org/10.33774/coe-2026-4f53g **ORCID:** https://orcid.org/0009-0007-9777-4393 Content not peer-reviewed by Cambridge University Press. **License:** CC0 1.0

[7] Varun Pratap Bhardwaj. *AgentAssay: Token-Efficient Regression Testing for Non-Deterministic AI Agent Workflows*. arXiv:2603.02601, 2026.

[8] Stephan Rabanser, Sayash Kapoor, Peter Kirgis, Kangheng Liu, Saiteja Utpala, and Arvind Narayanan. *Towards a Science of AI Agent Reliability*. arXiv:2602.16666, 2026.

[9] Shaina Raza, Ranjan Sapkota, Manoj Karkee, and Christos Emmanouilidis. *TRiSM for Agentic AI: A Review of Trust, Risk, and Security Management in LLM-based Agentic Multi-Agent Systems*. arXiv:2506.04133, 2025.

[10] R. A. Rasheed. *Claim-Level Auditability for Deep Research Agents*. arXiv:2602.13855, 2026.

[11] S. Sureshkumar. *R-LAM: Reproducibility-Constrained Large Action Models*. arXiv:2601.09749, 2026.

[12] World Wide Web Consortium. *PROV Overview and PROV Data Model*. W3C Recommendation, 2013.

[13] OpenTelemetry. *Traces*. OpenTelemetry documentation, 2026.

[14] Junde Wu. *Git Context Controller: Manage the Context of LLM-based Agents like Git*. arXiv:2508.00031, 2025.

[15] Xiwei Xu, Robert Mao, Quan Bai, Xuewu Gu, Yechao Li, and Liming Zhu. *Everything is Context: Agentic File System Abstraction for Context Engineering*. arXiv:2512.05470, 2025.

[16] Ian Stetsenko. *Lore: Repurposing Git Commit Messages as a Structured Knowledge Protocol for AI Coding Agents*. arXiv:2603.15566, 2026.

[17] J. Feng et al. *On the Effectiveness of Context Compression for Repository-Level Tasks: An Empirical Investigation*. arXiv:2604.13725, 2026.

[18] N. Verma et al. *Autonomous Memory Management in LLM Agents*. arXiv:2601.07190, 2026.

[19] W. Lyu et al. *Self-Organizing Multi-Agent Systems for Continuous Software Development*. arXiv:2603.25928, 2026.

[20] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. *SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering?*. arXiv:2603.15401, 2026.

[21] Alexandre Cristovao Maiorano. *Automated Self-Testing as a Quality Gate: Evidence-Driven Release Management for LLM Applications*. arXiv:2603.15676, 2026.

https://doi.org/10.33774/coe-2026-4f53g **ORCID:** https://orcid.org/0009-0007-9777-4393 Content not peer-reviewed by Cambridge University Press. **License:** CC0 1.0